Evidencia 2. Análisis del caso de éxito de Uber

Respuestas a las preguntas de discusión del caso
Oscar Cárdenas Valdez, A01741965
TC3009C, Inteligencia artificial avanzada para la ciencia de datos, Gpo 602
24 de septiembre de 2026

Las cifras que aparecen abajo salen del documento del caso, "Big data - Clase 1 - Caso Estudio Uber", preparado por Juan Antonio Peña González. Al final dejo una nota sobre dos números del caso que no cuadran entre sí.

1. Sobre el límite del escalamiento vertical y la consistencia ACID

> *Uber inició guardando todos sus viajes en una base relacional monolítica. ¿En qué punto operativo preciso las características de una base ACID pasan de ser una garantía de seguridad a convertirse en una amenaza para la disponibilidad del negocio?*

El punto exacto es cuando el ritmo al que llegan las escrituras supera el ritmo al que el nodo alcanza a confirmarlas en disco. Mientras las escrituras caben en el I/O de la máquina, los bloqueos y el mantenimiento síncrono de índices cuestan milisegundos y nadie los nota. En cuanto la cola de escrituras pendientes crece más rápido de lo que se vacía, la latencia deja de tener techo y el mismo bloqueo que protegía la consistencia es el que tira la disponibilidad.

En Uber ese cruce llega por la telemetría. El caso reporta cerca de 250 GB por hora por vehículo, alrededor de 2 TB diarios, y con cientos de miles de vehículos activos el perfil de carga es escritura continua sobre datos que casi nadie consulta en ese instante. Para una base relacional es el peor escenario posible, porque cada evento GPS obliga a mantener índices pesados pensados para consultas analíticas que en ese momento no se están haciendo.

Escalar hacia arriba deja de servir por dos razones distintas. Una es física: el cuello de botella es el I/O de disco y no el CPU, así que agregar núcleos no mueve la aguja. La otra es de calendario, entre 2010 y 2014 la curva de crecimiento de Uber iba más rápido que el ciclo de comprar, recibir e instalar hardware, y ninguna máquina se puede pedir con seis meses de anticipación a una demanda que se duplica sola.

El paso a BASE cambia la promesa. En vez de garantizar que toda lectura vea el último dato escrito, garantiza que el dato entre al sistema y que en una ventana acotada todos lo vean igual. Para una coordenada GPS eso es perfectamente aceptable, porque un mapa dos segundos atrasado no le cuesta dinero a nadie.

2. Sobre el teorema CAP y PACELC en la experiencia del usuario

> *Si durante un evento masivo ocurre una partición de red entre centros de datos de Uber, ¿por qué la empresa aplica un enfoque AP para la pantalla de mapas y asignación de viajes, pero un enfoque CP para la pasarela de pagos?*

Porque el costo de equivocarse no es el mismo de los dos lados, y la decisión se toma sobre ese costo antes que sobre una preferencia técnica.

Del lado del mapa, si durante la partición el conductor aparece dos segundos atrasado, el viaje se completa y nadie reclama. Si en cambio la aplicación responde con un error, la venta se pierde en ese mismo instante y el usuario abre la app de la competencia. Es una pérdida inmediata, medible y que no se recupera después.

Del lado de pagos la asimetría se invierte. Un cobro duplicado genera contracargo, ticket de soporte, un posible problema regulatorio y un daño de confianza que dura mucho más que el viaje. Hacer esperar al usuario o pedirle que reintente sale más barato que cobrarle dos veces, así que ahí conviene bloquear y reintentar.

PACELC agrega el caso que CAP deja fuera, el de operación normal sin partición, donde la elección es entre latencia y consistencia. Uber la resuelve igual de segmentada: para el mapa prefiere latencia, para el pago prefiere consistencia.

La consecuencia de diseño me parece lo más importante del caso. Uber no eligió una política global para toda la empresa, segmentó por tipo de transacción, y eso obliga a que la arquitectura sostenga los dos modos al mismo tiempo. Por eso el caso reporta Docstore con motores de consenso para las entidades financieras y almacenamiento de consistencia eventual para la geolocalización. Una sola base de datos no podía cumplir las dos promesas a la vez.

3. Sobre las 5 Vs del Big Data y el surgimiento de Apache Hudi

> *Si un Data Lake tradicional procesa archivos inmutables en bloque, ¿cómo rompe Hudi la restricción del procesamiento batch tradicional para introducir capacidades transaccionales sobre Big Data?*

Un data lake tradicional guarda archivos inmutables. Para cambiar un solo registro hay que reescribir la partición completa que lo contiene, de modo que actualizar el 1 por ciento obliga a mover el 100. Esa es la restricción que Hudi rompe, y la rompe agregando dos piezas que el lake no tenía.

La primera es un índice que relaciona la llave de cada registro con el archivo donde vive. Sin ese índice, localizar el 1 por ciento que cambió cuesta exactamente lo mismo que leerlo todo, y ahí se va la ventaja antes de empezar.

La segunda es una línea de tiempo de commits. Cada escritura queda registrada como una operación con su instante, y eso habilita dos cosas:

- El upsert. Se escribe solo el delta y la lectura resuelve cuál es la versión vigente, ya sea mezclando el archivo base con los deltas al momento de leer o consolidando al momento de escribir.
- La consulta incremental. Un consumidor pide únicamente lo que cambió desde cierto commit en lugar de barrer la tabla entera, que es lo que convierte un pipeline batch en uno casi continuo.

La medida del problema está en los números del caso. Antes de Hudi, reingesta completa de 120 TB cada 8 horas para aplicar menos de 500 GB de cambios. Después, escribir los registros modificados y actualizar punteros, con la ingesta e indexación bajando de horas a minutos.

Visto de lejos, Hudi mete garantías transaccionales sobre un almacenamiento que nunca las tuvo, sin obligar a regresar a una base relacional que ya había demostrado que no aguantaba el volumen.

4. Debrief: la regla de oro de la infraestructura de datos

La regla que me llevo del caso es ésta: la arquitectura de datos se elige por el costo de equivocarse en cada operación, no por un estándar técnico único para toda la empresa.

Uber no migró todo a NoSQL ni conservó todo en relacional. Clasificó sus operaciones según lo que pasa cuando el dato llega tarde o llega mal, y sobre esa clasificación eligió tecnología. Donde equivocarse cuesta una venta perdida, eligió disponibilidad. Donde equivocarse cuesta un cobro duplicado, eligió consistencia. La misma lógica explica por qué inventó Hudi en lugar de comprar algo: el costo de reescribir 120 TB para cambiar menos de 500 GB era tan alto que justificaba construir una pieza nueva.

Los números del caso sostienen la regla. Las decisiones que más ahorraron son las que estaban alineadas con el costo de error de su dominio. CacheFront v2 baja el consumo de 60,000 a 3,000 núcleos, un 95 por ciento menos, mientras multiplica las lecturas atendidas de 6 millones a más de 150 millones por segundo. La arquitectura UFA baja el aprovisionamiento de 2x a 1.3x, que es CAPEX que deja de gastarse en servidores ociosos.

5. Debrief: lo que aprendí sobre la descomposición estratégica del negocio

La descomposición útil es por dominio de negocio y no por capa técnica. DOMA agrupa los microservicios por dominio, y eso logra dos cosas al mismo tiempo: contiene las fallas dentro de un dominio en vez de propagarlas, y deja que cada dominio elija su propio modelo de consistencia y su propio almacenamiento sin negociar con el resto de la empresa.

Descomponer también crea un problema nuevo que hay que resolver en el mismo movimiento. Cuando un clic dispara llamadas encadenadas a través de decenas de servicios, la trazabilidad deja de ser opcional. Sin ella lo que queda es un monolito distribuido, que junta las desventajas de los dos modelos. Uber desarrolló Jaeger justo para eso, y el caso lo ubica en el mismo periodo de la transición a microservicios, 2015 a 2016, no como algo que se agregó años después.

El efecto es financiero además de técnico. Separar dominios es lo que hace posible la conmutación activa-activa entre regiones, y eso es lo que permite bajar la redundancia pasiva de 2x a 1.3x. Un monolito no se puede repartir así, tiene que replicarse entero.

Conexión con el reto de Ternium

Nuestro equipo trabaja el Reto A, oferta inteligente de viajes y ranking de transportistas. Las dos ideas del caso que aplican directo son la segmentación por costo de error y la consulta incremental.

Por costo de error, el ranking de transportistas y la sugerencia de oferta toleran consistencia eventual sin problema: si el ranking se calculó con datos de hace cinco minutos, la recomendación sigue siendo útil. La tarifa pactada y el registro del viaje adjudicado son otra cosa y necesitan consistencia estricta, porque ahí un dato duplicado o perdido es dinero mal facturado a un proveedor.

Por consulta incremental, el historial de viajes y tarifas es una tabla que se consulta mucho y se corrige poco, que es exactamente el patrón para el que sirve un upsert sobre almacenamiento inmutable en lugar de reprocesar el histórico completo cada vez que se ajusta una tarifa.

Nota sobre dos cifras del caso

Al verificar los números encontré dos puntos que no cuadran entre sí dentro del documento fuente, y los dejo anotados porque cambian cómo se citan:

- El caso reporta cerca de 250 GB por hora por vehículo y los resume como 2 TB diarios. A 24 horas serían unos 6 TB; la cifra de 2 TB corresponde a una jornada de alrededor de 8 horas de operación. Usé 2 TB diarios porque es la cifra que el documento fija.
- La sección de tecnologías y la matriz de impacto dicen que se reingerían 120 TB completos cada 8 horas, mientras que la pregunta 3 lo enuncia como 120 TB diarios. Tomé la primera versión, que aparece dos veces y es la que hace consistente la comparación contra los menos de 500 GB de cambios.

Fuente

Peña González, J. A. (2026). *Big data - Clase 1 - Caso Estudio Uber*. TC3009C, Tecnológico de Monterrey. Documento de clase.